stylometric-transfer

> **Stylometric profiling + controllable author-style transfer for personal writing**

`stylometric-transfer` constructs an explicit, interpretable **stylometric style profile** from an author's corpus, then applies that profile to rewrite or generate new text in the same voice.

In practical terms, the system enables an LLM to apply a specified writing style to any text input. It performs stylometric profiling and humanization on writing samples, then uses constraint-guided author-style transfer for a target document. A style "fingerprint" is built from the writing corpus using classic stylometric measurements and graph structure, and that fingerprint is applied via an LLM to rewrite any text.

Unlike fine-tuning or opaque embeddings, the system uses an **explicit, versionable JSON style model** that can be inspected, edited, audited, and reused. A **humanization-aware conflict-resolution layer** integrates humanizer guidelines directly into the rewrite step, without violating the fingerprint's style constraints.

This repository includes:

- `fingerprint_style.py` : extracts a **style fingerprint (stylometric profile)** from a writing archive
- `apply_fingerprint.py` : rewrites Markdown to match the fingerprint
- `fingerprint_api.py` : local HTTP API exposing `make` , `apply` , `rate` , and `similarity` methods
- `fingerprint_api_harness.py` : GUI demo harness for calling API endpoints
- `show_fingerprint.py` : generates a standalone HTML dashboard for a fingerprint JSON
- `common.py` : shared path/config/store/probability helpers used across entry points
- `prompts.json` : externalized prompt templates used by both scripts (edit here to adjust behaviour)
- `api/swagger/openapi.yaml` and `api/swagger/openapi.json` : API specifications
- `scripts/` : bash wrappers for invoking the Python entry points (including `fingerprint_api_harness.sh`)

Further details are available in `Article-Teaching-Machines-to-Write-Like-You.md` and `Research-Paper.md` .

Comments/contributions are encouraged and appreciated.

License

PolyForm Noncommercial License 1.0.0. This project is licensed under the **PolyForm Noncommercial License 1.0.0** (see `LICENSE.md`).

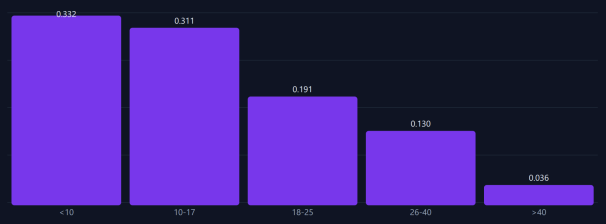
Key points:

- Noncommercial only: Use, modification, and redistribution are permitted for noncommercial purposes.
- Commercial use requires permission: Any commercial use (including paid products or services incorporating this code) requires explicit permission from the author.
- Attribution required: Redistribution or use of substantial portions of this project must include clear credit and preserve the license/notice requirements described in `LICENSE.md`.

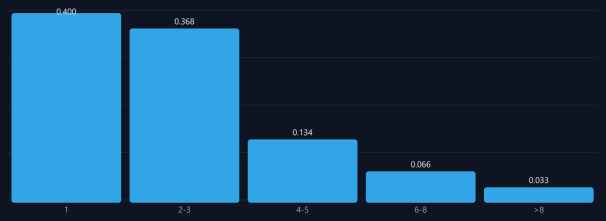
For commercial use, contact the author to discuss potential participation and/or licensing.

Style Fingerprint Dashboard

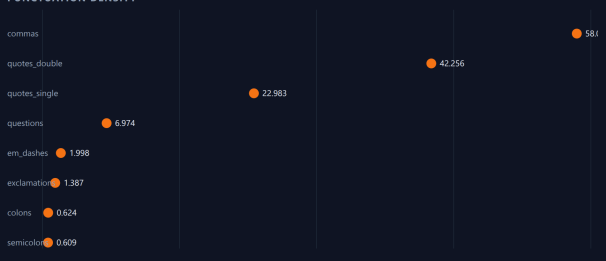
SENTENCE LENGTHS



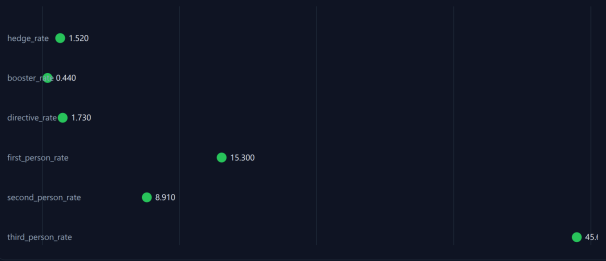
PARAGRAPH LENGTHS



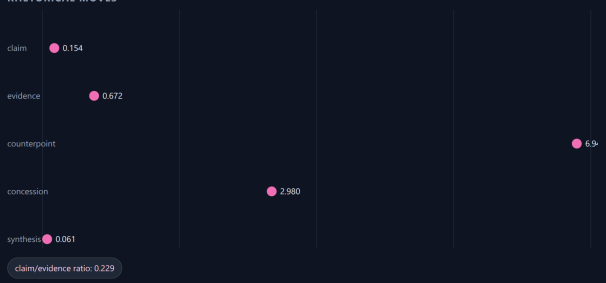
PUNCTUATION DENSITY



STANCE SIGNALS



RHETORICAL MOVES



OVERVIEW

Profile	[redacted]
Schema	1.0.0
Author	[redacted]
Self	[redacted]
Source	[redacted]

CORPUS TOTALS

Documents	4
Words	538578
Sentences	33663
Paragraphs	14433

WORKS

- [redacted]
- [redacted]
- [redacted]
- [redacted]

FUNCTION WORDS

the	35,240
to	14,442
a	13,072
and	12,038
of	11,213
was	8,818
he	7,786
in	7,349
that	6,422
it	5,530

LEXICON - PREFER

technical terms · military jargon · precise nouns · direct verbs · claim · evidence · counterpoint · concession · synthesis · directive · speculative · probabilistic · assertive

LEXICON - RARE WORDS

allocated · arounds · beeped · bombshell · carrot · clung · dinosaur · distrust · eerily · entrenched · facade · feasting · garishly · geologist · hammering · harnessing · impaling · inference · jetway · jockey · kickers · knowin · limitless · loaned · masterpiece · melange · nervewrenching · nonregulars · ominously · organizers · prepares · prey · questing · quizzed · rays · reheated · salsa · scourge · throb · tinkling · unblinking · unplug · vanish · vapid · wobbling · workarounds · xeroxed · xiansheng · yachts · yield · zealous · zipper

LEXICON - AVOID (SOFT)

april · august · lol · etc · album · trump · obama · songs · cent · twitter · fund · louis · paris · http · richard · dont · blog · festival · links · prince · labour · ben · artists · elections · debate · context · examples · voting · solo · scottish · shares · statistics · wales · dreams · gender · kim · rural · scenes · andrew · bbc · clinton · competitive · favourite · programme · crown · kevin · pregnant · tournament · ban · experiences · legislation · democrats · multi · nfl · fees · haha · manchester · princess · debut · fantasy · joseph · seasons · singer · thats · theatre · comedy · defence · lets · trend · champions · republicans · stephen · chocolate · coal · colour · criticism · historic · settlement · grey · guitar · lewis · sarah · everyday · cats · donald · actress · charity · formula · luke · publication · rating · rome · expectations · muslims · wage · commissioner · liverpool · opens · romantic · welfare

TEMPLATES - OPENERS

There was no [noun]... · What do you [verb]... · He didn't know [clause]... · A moment later, [clause]... · You know how [clause]... · In any case, [clause]... · there was no · what do you

TEMPLATES - TRANSITIONS

First, [clause]... · Finally, [clause]... · Second, [clause]... · Therefore, [clause]... · However, [clause]... · first · finally · second

LEXICON - AVOID (HARD)

None

CONTROLS

Rewrite policy	preserve technical, military, and procedural tone; avoid informal slang, emotional adjectives, and intensifiers; preserve technical and procedural details. Avoid informal slang except in dialogue; avoid intensifiers and emotional adjectives unless in dialogue. Preserve technical detail and narrative rhythm; avoid introducing informal or emotional language. Preserve narrative structure and technical detail; avoid informal slang and emotional adjectives.
Priority order	technical accuracy, procedural clarity, narrative realism, lexical precision, templates, paragraph_cadence, syntax_texture, rhetoric_moves, Maintain technical precision and realism, Favor evidence and detail over direct assertion, Use counterpoint and concession liberally, Avoid intensifiers, emotional adjectives, and informal slang, lexicon, templates, syntax_texture, paragraph_cadence, rhetoric_moves, epistemic_profile, lexical, syntactic, rhetorical, paragraph rhythm

STRICTNESS

Strictness	medium

VALIDATOR WEIGHTS

orthography	0.10
punctuation	0.10
sentence_length	0.15
paragraph_length	0.15
function_words	0.10
stance_signals	0.10
templates	0.10
rhetoric_moves	0.20
epistemic_profile	0.10
syntax_texture	0.15
lexicon	0.25
lexical_avoidance	0.10
paragraph_cadence	0.15
repetition	0.10
discourse_markers	0.05

EXTRACTION

Model	[redacted]
Date	2024-06-10
Confidence	high

PERSONA

Default pronouns	he/him
Allowed	he/him, she/her, they/them
Avoid	
Strictness	soft

An example fingerprint (output from show_fingerprint.py)

Table of Contents

- Overview
- Concepts & Terminology
- Features
- Architecture
- Installation
- Configuration
- Usage
  - 1. Build a Style Fingerprint
  - 2. Apply a Fingerprint
  - 3. Local HTTP API
  - 4. API GUI Harness
- Output Files
- Testing
- Style Model Schema
- Ethics & Intended Use
- Roadmap
- References

Overview

The project implements a two-stage pipeline:

1. **Stylometric profiling**
   Quantitative analysis of an author's corpus to construct a structured **style fingerprint** (JSON)

2. **Author-conditioned style transfer**
   Rewriting new text to conform to that fingerprint while preserving meaning

The system integrates:

- Local statistical measurement (sentence length, punctuation rates, paragraph structure, etc.)
- LLM-based synthesis into an explicit style model
- Constraint-driven rewriting using that model

This is a practical implementation of:

> **Stylometric profiling + controlled author-style transfer**

Concepts & Terminology

Term	Meaning
Stylometry	Quantitative analysis of writing style (measurable signals, not semantics).
Stylometric profile	A feature-based summary of an author's style derived from a corpus.
Style fingerprint	The explicit JSON artifact that encodes style measurements, targets, and controls.
Style transfer	Rewriting text to match a fingerprint while preserving meaning.
Author-conditioned generation	Generating new text guided by a fingerprint rather than a raw prompt alone.
Measurements	Observed statistics from the corpus (e.g., sentence length histograms, punctuation rates).
Targets	Desired ranges or qualitative goals derived from measurements (used in rewriting).
Lexicon	Preferred/avoided words and phrases; soft or hard constraints.
Templates	Rhetorical or syntactic patterns (openers, transitions, paragraph moves).
Controls	Priority/strictness and rewrite policies that govern tradeoffs.
Validators	Checks and weights used to score compliance or detect deviations.
Deviations	Structured report of where the model could not comply or had to adjust.
Control normalization	Deterministic de-duplication of `rewrite_policy` clauses and token filtering for `priority_order`.
Humanizer rules	General guidelines (from `general-guidelines.md`) filtered for conflicts with the fingerprint.
Tunables	Runtime configuration (`config.tunables.json`) that shapes filtering, retries, chunking, and metrics.
Entity blacklist	Names/places/orgs list used to suppress proper-name phrases during phrase validation.
Lexical avoidance list	Common words the author rarely uses (treated as soft avoids).

Term	Meaning
Fiction vs non-fiction	Classification that determines whether multi-word quotes are rewritten or preserved.
Chunking	Splitting large inputs to fit the model context, then reconciling outputs.
Style retry	Optional delta-feedback loop that re-prompts if compliance score is low.
Humanization metrics	Quantitative, research-grounded signals used to score “human-likeness.”

In research terms, the system performs:

- Feature-based stylometric profiling from real corpus statistics
- Interpretable, constraint-driven rewriting/generation
- Conflict-aware humanization aligned with explicit style constraints

Features

- Accepts `.zip` / `.tar*` archives of writing corpora
- Reads `.txt`, `.md`, `.rst`, `.html`, `.docx` (via `python-docx`)
- Computes statistical measurements locally:
  - Sentence length distributions
  - Paragraph structure
  - Punctuation rates
  - Contraction and dash usage
  - US vs Canadian spelling heuristic (English-only)
  - Common n-grams
  - Function-word profile and stance signals (hedging/boosting/pronouns)
  - Sentence-opener and transition templates (top patterns)
  - Rare-word signals (words the author rarely uses)
  - One-sentence paragraph rate / paragraph rhythm
  - Rhetorical move rates (claim/evidence/counterpoint/concession/synthesis)
  - Paragraph cadence (opening/closing sentence length stats)
  - Epistemic stance bands (speculative/probabilistic/assertive/directive)
  - Syntax texture (subordinate/parenthetical/appositive rates)
  - Discourse marker positions (start vs mid-sentence)
  - Repetition signals (bigram/trigram repeat rates)
- Produces a **comprehensive JSON style profile**
- Rewrites Markdown with:
  - Meaning preservation
  - Structural fidelity
  - Deviation reporting
  - Optional style-compliance retry with delta feedback
  - Deterministic normalization of verbose rewrite policies and noisy priority orders before use
- Filters out blockquotes, reference sections, footnotes, citation markers, and boilerplate notices (copyright/terms/privacy) from style measurements and excerpts, preserving them verbatim during rewrite
- Strips embedded BASE64 images before sending prompts to the LLM and re-embeds them in output
- Exposes a local HTTP API (`make` / `apply` / `rate`) backed by GUID-tracked fingerprint files
  - Includes a fingerprint-to-fingerprint `similarity` method for profile comparison diagnostics
- Compatible with OpenAI (works with OpenAI, Azure OpenAI, vLLM, etc.)
- Interpretable, editable, versionable style models

Architecture

```
corpus.tar.gz
      │
      ▼
[fingerprint_style.py]
      │
      ├─ local statistical analysis
      ├─ representative excerpts
      └─ LLM synthesis
      │
      ▼
style_fingerprint.json
      │
      ▼
[apply_fingerprint.py]
      │
      ├─ local measurement of input
      ├─ constraint-driven rewriting
      └─ deviation audit
      │
      ▼
rewritten_text.styled.md

HTTP text
      │
      ▼
[fingerprint_api.py]
      │
      ├─ POST /make  -> fingerprint_style.py + GUID store
      ├─ POST /apply -> apply_fingerprint.py
      └─ POST /rate  -> local probabilistic style scoring
         POST /similarity -> local fingerprint similarity scoring
```

Installation

Requirements

- Python 3.9+
- `requests`
- `python-docx` (optional, for `.docx` corpora)

```
pip install requests python-docx
```

Configuration

Create `config.llm.json` in the project root (used by default):

```
{
  "api_key": "YOUR_OPENAI_KEY",
  "base_url": "https://api.openai.com/v1",
  "model": "gpt-4.1-mini",
  "max_tokens": 6000,
  "max_prompt_tokens": 6000,
  "temperature": 0.2,
  "timeout_seconds": 300,
  "max_retries": 6,
  "backoff_base_seconds": 2.0,
  "backoff_max_seconds": 20.0
}
```

Notes:

- Default lookup for `config.llm.json`: current working directory first, then the directory containing the Python scripts
- Optional `config.llm.roster.json` (same lookup path) defines ordered model entries used by `--roster` (one model per chunk, cycling through the roster)
- `config.tunables.json` can override humanizer conflict thresholds (same search path as config.llm.json)
- `fingerprint_api.py` uses the same config search behavior as the CLI tools and forwards settings to existing entry points (`fingerprint_style.py`, `apply_fingerprint.py`)
- `max_prompt_tokens` controls chunking for large inputs (defaults to `max_tokens`; override per run with `--max-prompt-tokens`)

- `max_retries`, `backoff_base_seconds`, and `backoff_max_seconds` control exponential backoff retries for transient LLM errors or timeouts
- `base_url` should be the API root (no `/chat/completions`)
- Any OpenAI-compatible endpoint can be used
- Lower temperature is recommended for consistency
- Prompt templates are stored in `prompts.json` next to the Python scripts and are loaded at runtime (includes the `validate_phrases` template used for common-phrase validation)
- Optional `lexicon_hints.json` (in repo root or next to the scripts) can provide preferred or avoided phrases for fingerprinting
- Optional `config.avoid.txt` (in repo root or next to the scripts) lists words or phrases to always avoid; it is merged into the fingerprint lexicon and enforced during style application
- Optional `config.common_words.txt` (in repo root or next to the scripts) defines the common-word list used to derive `measurements.lexical_avoidance.rare_words` (words common in English but absent from the corpus). Format: `word <zipf_frequency>`; the frequency is optional but used to prioritize higher-frequency words first.
- Optional `config.entity_blacklist.txt` (in repo root or next to the scripts) lists entities (people, places, organizations) to suppress from common-phrase extraction to avoid proper-name dominance
- Genre handling: by default the tools auto-detect fiction vs non-fiction; override with `--fiction` or `--non-fiction`. In non-fiction, multi-word quotations are excluded from fingerprinting and preserved verbatim during rewriting.
- When fingerprinting, the current `config.tunables.json` can be embedded as `metadata.extraction.tunables_snapshot` for auditability
- If the fingerprint prompt exceeds `max_prompt_tokens`, excerpts are chunked and **partial fingerprints are merged using a second LLM merge pass** (pairwise merge with a dedicated merge prompt).
- Common-phrase validation now includes a deterministic prefilter (honorifics + capitalization-ratio heuristics, entity blacklist, date patterns) and an optional LLM phrase-ranking step that drops likely proper-name phrases before final selection.
- Rare-word selection can be ranked by the same LLM validation call used for common phrases, to de-prioritize proper names before truncation.
- **Corpus size guidance:** diminishing returns typically appear once core style statistics stabilize. As a rule of thumb, ~20–50k words often yields a stable fingerprint for a single author/genre; ~100k words usually captures most steady signals. If key rates (sentence/paragraph distributions, punctuation per 1k words, function-word profile, stance rates) drift by <1–2% after adding another 10–20k words, you're likely in the diminishing-returns zone. More data still helps when you're mixing genres/eras or chasing rare rhetorical/lexical signals.

Tunables: `config.tunables.json`

`apply_fingerprint.py` uses `config.tunables.json` to determine which humanizer guidelines conflict with the fingerprint or the input Markdown style. Any rule that conflicts is dropped before prompting. `fingerprint_style.py` can also embed a `tunables_snapshot` under `metadata.extraction` to preserve the exact tunables used during profile creation.

Example (`config.tunables.json` project configuration example):

```
{
  "humanizer_conflicts": {
    "em_dash_keep_rate": 0.5,
    "hedge_keep_rate": 1.0,
    "first_person_keep_rate": 0.5,
    "contractions_avoid_threshold": 2.0,
    "contractions_use_threshold": 0.5,
    "heading_title_case_keep_rate": 0.6,
    "boldface_keep_per_1000w": 3.0,
    "inline_header_list_keep_rate": 0.2
  },
  "humanizer_mandatory": {
    "avoid_em_dashes": true,
    "emoji_policy": "replace",
    "normalize_double_quotes": true,
    "normalize_single_quotes": true,
    "heading_case_normalization": "by-level",
    "heading_case_by_level": {
      "h1": "title-case",
      "h2": "title-case",
      "h3": "title-case",
      "h4": "sentence-case",
      "h5": "identical",
      "h6": "automatic",
      "h7": "lower",
      "h8": "upper"
    },
    "preserve_proper_name_case": true,
    "sanitize_heading_qualifiers": {
      "enabled": true,
      "allowlist": ["quick win"]
    },
    "force_local_spelling_LLM": "none",
    "force_local_spelling_rules": "canadian"
  },
  "perplexity_level": "default",
  "perplexity_profiles": {
    "default": {
      "humanizer_variance": { "max_ops_per_1000w": 0.5 },
      "humanization_controller": { "quantiles": [0.25, 0.5, 0.75], "range_pct":
0.15 },
      "chunking": { "max_input_tokens": 5750, "min_chunks_when_perturbing": 2
```

```
},
      "llm": { "temperature_multiplier": 1.0 }
    },
    "low": {
      "humanizer_variance": { "max_ops_per_1000w": 1.0 },
      "humanization_controller": { "quantiles": [0.2, 0.5, 0.8], "range_pct":
0.2 },
      "chunking": { "max_input_tokens": 5200, "min_chunks_when_perturbing": 3
},
      "llm": { "temperature_multiplier": 1.0 }
    },
    "medium": {
      "humanizer_variance": { "max_ops_per_1000w": 1.5 },
      "humanization_controller": { "quantiles": [0.15, 0.5, 0.85], "range_pct":
0.25 },
      "chunking": { "max_input_tokens": 4700, "min_chunks_when_perturbing": 4
},
      "llm": { "temperature_multiplier": 1.0 }
    },
    "high": {
      "humanizer_variance": { "max_ops_per_1000w": 2.0 },
      "humanization_controller": { "quantiles": [0.1, 0.5, 0.9], "range_pct":
0.3 },
      "chunking": { "max_input_tokens": 4200, "min_chunks_when_perturbing": 5
},
      "llm": { "temperature_multiplier": 1.0 }
    },
    "extreme": {
      "humanizer_variance": { "max_ops_per_1000w": 2.0 },
      "humanization_controller": { "quantiles": [0.1, 0.5, 0.9], "range_pct":
0.3 },
      "chunking": { "max_input_tokens": 4200, "min_chunks_when_perturbing": 5
},
      "llm": { "temperature_multiplier": 2.0 }
    }
  },
  "humanizer_variance": {
    "enabled": true,
    "seed": 12345,
    "max_ops_per_1000w": 0.5,
    "allowed_ops": ["swap_transition", "drop_filler"]
  },
```

```
  "humanization_metrics": {
    "weights": {
      "lexical_diversity": 1.0,
      "herdan_c": 1.0,
      "guiraud_r": 1.0,
      "maas_ttr_inverse": 1.0,
      "yules_k_inverse": 1.0,
      "simpson_d_inverse": 1.0,
      "repetition_inverse": 1.0,
      "sentence_burstiness": 1.0,
      "paragraph_burstiness": 1.0,
      "punctuation_variety": 1.0,
      "punctuation_entropy": 1.0,
      "function_word_entropy": 1.0,
      "function_word_kl_inverse": 1.0,
      "sentence_length_js_inverse": 1.0,
      "char_trigram_entropy": 1.0,
      "avg_word_length": 1.0
    }
  },
  "humanization_baseline": {
    "enabled": true,
    "window_words": 800,
    "stride_words": 400,
    "min_window_words": 250,
    "max_windows": 200
  },
  "humanization_controller": {
    "enabled": true,
    "seed": 12345,
    "quantiles": [0.25, 0.5, 0.75],
    "range_pct": 0.15,
    "min_width": 0.05,
    "max_width": 6.0,
    "allowed_metrics": [
      "sentence_length_mean",
      "sentence_length_stdev",
      "one_sentence_paragraph_rate",
      "comma_density_per_100w",
      "punctuation_semicolons_per_1000w",
      "punctuation_colons_per_1000w",
      "punctuation_em_dashes_per_1000w"
```

```
    ],
    "feedback_enabled": true,
    "feedback_tolerance": 0.35,
    "max_feedback_retries": 3
  },
  "lexical_signals": {
    "rare_words_limit": 100
  },
  "lexical_avoidance": {
    "rare_words_limit": 100
  },
  "controls_normalization": {
    "rewrite_policy": {
      "jaccard_threshold": 0.6,
      "dedupe_on_subset": true,
      "prefer_more_specific": true,
      "compress_directives": true,
      "directive_verbs": [
        "preserve",
        "avoid",
        "maintain",
        "ensure",
        "keep",
        "favor",
        "use",
        "prefer",
        "minimize",
        "maximize",
        "do not",
        "don't"
      ],
      "stopwords": [
        "the",
        "and",
        "of",
        "to",
        "a",
        "an",
        "in",
        "on",
        "for",
        "with",
```

```
        "or",
        "but",
        "as",
        "by",
        "from",
        "into",
        "at",
        "that",
        "this",
        "these",
        "those",
        "be",
        "is",
        "are",
        "was",
        "were",
        "been",
        "being"
      ]
    },
    "priority_order": {
      "token_pattern": "^[A-Za-z][A-Za-z0-9_\\-]*$",
      "dedupe_case_insensitive": true,
      "exclude_tokens": ["lexical", "syntactic", "rhetorical"]
    }
  },
  "fiction_detection": {
    "quote_span_min": 6,
    "quoted_ratio_min": 0.03,
    "quote_para_ratio_min": 0.2,
    "quoted_ratio_force": 0.08
  },
  "chunking": {
    "max_input_tokens": 5750,
    "chunk_split_on": "sentence",
    "chunk_summary": {
      "enabled": true,
      "summary_words": 50
    },
    "min_chunks_when_perturbing": 2,
    "recovery_split_max_depth": 2,
    "recovery_split_min_chars": 800,
```

```
    "variance_aware": {
      "enabled": true,
      "sentence_stdev_ref": 18.0,
      "paragraph_burst_ref": 0.7,
      "min_factor": 0.6,
      "max_factor": 1.0
    }
  },
  "style_retry": {
    "enabled": true,
    "threshold": 0.60,
    "max_retries": 2,
    "voice_max_retries": 2
  },
  "section_restore": {
    "enabled": true,
    "max_restore_sections": 20,
    "heading_similarity_threshold": 0.5,
    "signature_similarity_threshold": 0.35,
    "signature_min_overlap": 6
  },
  "postprocess_redundancy": {
    "enabled": true,
    "paragraph_dedupe": {
      "enabled": true,
      "min_words": 30,
      "similarity_threshold": 0.985,
      "lookback_blocks": 20,
      "max_drop_ratio": 0.15
    },
    "list_density": {
      "enabled": true,
      "min_run_length": 9,
      "group_size": 2,
      "joiner": "; "
    }
  },
  "sanity_checks": {
    "line_count_warn_pct": 10.0,
    "word_count_warn_pct": 10.0,
    "paragraph_count_warn_pct": 10.0
```

```
  }
}
```

Explanation of each tunable (grouped by theme)

Humanizer Conflict Thresholds (`humanizer_conflicts`) *These thresholds decide when humanizer guidance is considered contradictory to measured author style and should be dropped.*

- `em_dash_keep_rate` (per 1000 words): if the fingerprint's em-dash rate is **at or above** this value, the "avoid em dashes" guideline is considered conflicting and removed.
- `hedge_keep_rate` (per 1000 words): if the fingerprint's hedging rate is **at or above** this value, "avoid hedging" guidance is dropped.
- `first_person_keep_rate` (per 1000 words): if the fingerprint's first-person rate is **below** this value (or pronoun preferences avoid first-person), "use I/first-person" guidance is dropped.
- `contractions_avoid_threshold` (per 1000 words): if the fingerprint's contraction rate is **at or above** this value, any "avoid contractions" guideline is dropped.
- `contractions_use_threshold` (per 1000 words): if the fingerprint's contraction rate is **below** this value, any "use contractions" guideline is dropped.
- `heading_title_case_keep_rate` (0–1): if the input Markdown's headings are **mostly Title Case** (ratio at or above this value), the "avoid Title Case" guideline is dropped.
- `boldface_keep_per_1000w` (per 1000 words): if the input uses boldface **at or above** this density, "avoid boldface" guidance is dropped.
- `inline_header_list_keep_rate` (0–1): if the input uses inline-header list style (e.g., `- **Label:** text`) **at or above** this ratio, the "avoid inline-header lists" guideline is dropped.

Mandatory Deterministic Guards (`humanizer_mandatory`) *These are hard post-processing rules that apply regardless of stylistic variability.*

- `avoid_em_dashes` (boolean): when true, em-dashes are always removed in the output regardless of other signals.
- `emoji_policy` (`remove`, `replace`, or `none`): remove emojis, replace common ones with conventional monochrome symbols, or disable emoji handling.
- `normalize_double_quotes` (boolean): when true, curly double quotes are normalized to straight quotes.
- `normalize_single_quotes` (boolean): when true, curly single quotes are normalized to straight apostrophes. Backticks (including Markdown code ticks like `` ` `` and ``` `` ```) are not changed.
- `sanitize_heading_qualifiers` (boolean or object): when true (or enabled), trailing parenthetical/comma qualifiers in headings are removed if the remaining title still has at least two words.
- `sanitize_heading_qualifiers.enabled` (boolean): turn the qualifier sanitizer on/off.
- `sanitize_heading_qualifiers.allowlist` (array of regex strings): headings that match any pattern are exempt from qualifier stripping.

- `force_local_spelling_LLM` (`none` , `canadian` , `australian` , `british` , `us`): locale spelling instruction sent to the LLM. `none` sends no explicit locale spelling instruction.
- `force_local_spelling_rules` (`none` , `canadian` , `australian` , `british` , `us`): locale used by deterministic code-side normalization rules after generation.
- `force_local_spelling` (`none` , `canadian` , `australian` , `british` , `us`): legacy fallback. If split settings are absent, this value is used for both LLM and deterministic rules.

Perplexity Presets (`perplexity_level` , `perplexity_profiles`) *These presets provide one-switch variability tuning by overriding a small set of bounded knobs.*

- `perplexity_level` (`default` , `low` , `medium` , `high` , `extreme`): selected preset for the run.
- `perplexity_profiles.<level>` : per-level overrides applied to:
  - `humanizer_variance.max_ops_per_1000w`
  - `humanization_controller.quantiles`
  - `humanization_controller.range_pct`
  - `chunking.max_input_tokens`
  - `chunking.min_chunks_when_perturbing`
  - `llm.temperature_multiplier` (multiplies `config.llm.json` temperature; effective temperature is clamped to `0.0..2.0`)
- `default` should mirror your baseline settings; `low` / `medium` / `high` progressively increase variability and chunk-level perturbation opportunity; `extreme` adds aggressive model sampling by doubling base temperature.

Heading Case Normalization (`humanizer_mandatory.*`) *These settings define deterministic heading-case policy, either globally or by heading level.*

- `heading_case_normalization` (`automatic` , `identical` , `by-level`):
  - `automatic` : do not apply deterministic heading-case normalization.
  - `identical` : restore heading case from the source heading.
  - `by-level` : apply per-level heading case policy from `heading_case_by_level` .
- `heading_case_by_level` (object, used when mode is `by-level`): per-level policy with keys `h1` .. `h8` . Allowed values:
  - `automatic` (no deterministic rewrite)
  - `identical` / `unchanged` (restore source casing for that level)
  - `title-case` , `sentence-case` , `caps` (or alias `upper`), `lower`
- `preserve_proper_name_case` (boolean): when true, deterministic heading-case transforms preserve detected proper-name casing from the source heading (for example, `John Black` remains `John Black` even if the level policy is `caps`).
- Deterministic heading-case handling (either `identical` , or `by-level` with at least one non- `automatic` level) causes heading-style humanizer rules (for example, “Title Case in Headings”) to be dropped and logged as deterministic conflicts.

Stochastic Humanizer Perturbations (`humanizer_variance`) *These controls bound randomness so variation is deliberate, reproducible, and limited.*

- `humanizer_variance.enabled` (boolean): enables bounded stochastic micro-variation during application.
- `humanizer_variance.seed` (integer): RNG seed for deterministic runs.
- `humanizer_variance.max_ops_per_1000w` (float): maximum number of micro-operations per 1000 words. **Recommendation:** start at `0.5`; `0.5–1.5` is usually safe. Values above `2.0` can begin to feel noisy unless the input is highly repetitive.
- `humanizer_variance.allowed_ops` (array): allowed micro-operations (e.g., `swap_transition`, `drop_filler`). **Recommendation:** begin with `["swap_transition", "drop_filler"]`, add ops gradually, and keep the list short to avoid compounding randomness.
- `swap_transition`: swaps a transition phrase with another compatible transition to vary surface rhythm without changing meaning.
- `drop_filler`: removes low-information filler words/phrases when safe (bounded by the ops budget).

Humanization Scoring (`humanization_metrics`) *These weights control how individual humanization metrics contribute to the aggregate score.*

- `humanization_metrics.weights` (object): optional weighting for the 0–100 aggregate humanization score. Any metric with a weight of 0 is excluded.

Baseline Extraction (`humanization_baseline`) *These parameters govern rolling-window extraction of corpus-native variability baselines.*

- `humanization_baseline.enabled` (boolean): when true, fingerprinting computes rolling "within-author variability" baselines (stored under `measurements.humanization_baseline`). These baselines are for auditability/controller logic and are stripped from what the LLM sees during rewriting.
- `humanization_baseline.window_words` (integer): rolling window size (in words) used to compute baseline variability stats.
- `humanization_baseline.stride_words` (integer): stride size (in words) between windows.
- `humanization_baseline.min_window_words` (integer): minimum usable window size; if a window is smaller, baseline computation stops early.
- `humanization_baseline.max_windows` (integer): cap on how many windows are computed (keeps runtime bounded on very large corpora).

Controller Overlay (`humanization_controller`) *These settings shape per-chunk target overlays that nudge output toward baseline variation bands.*

- `humanization_controller.enabled` (boolean): enables per-chunk target overlays derived from the baseline (embedded in fingerprint, stripped from LLM prompt except as derived overlay targets).
- `humanization_controller.seed` (integer): deterministic seed for overlay sampling.
- `humanization_controller.quantiles` (array of 0–1): which baseline quantiles are eligible when sampling per-chunk targets (e.g., `[0.25, 0.5, 0.75]`).

- `humanization_controller.range_pct` (float): width of the target range around the sampled value (percentage of the value).
- `humanization_controller.min_width` (float): minimum absolute width for a target range.
- `humanization_controller.max_width` (float): maximum absolute width for a target range.
- `humanization_controller.allowed_metrics` (array): which overlay metrics are used (e.g., `sentence_length_mean`, `sentence_length_stdev`, `one_sentence_paragraph_rate`, `comma_density_per_100w`, `punctuation_semicolons_per_1000w`).
- `humanization_controller.feedback_enabled` (boolean): when true, style retry feedback includes overlay-mismatch guidance.
- `humanization_controller.feedback_tolerance` (float): how far outside the overlay range the output must be before controller feedback is added (fraction of range).
- `humanization_controller.max_feedback_retries` (integer): cap on how many style-retry passes include controller-overlay feedback. This does not create extra retries by itself.

Increasing output variability ("perplexity")

This project does not compute classic language-model perplexity directly. In practice, when users ask for "higher perplexity" they usually mean: the output feels less uniform and less template-like (more natural variation in rhythm, punctuation, transitions, and local phrasing) without changing meaning.

Quick preset option:

- Set `perplexity_level` in `config.tunables.json`, or pass `--perplexity {default|low|medium|high|extreme}` for a one-run override.
- Normal output includes the active mode/perplexity in the compact run header; verbose output prints the effective tuning values.

Recommended workflow:

1. Measure first (so you can see whether changes help).
    - Run `apply_fingerprint.py ... --metrics -v` and compare the printed humanization metrics and aggregate 0-100 score for input vs output.
2. Increase bounded stochastic variation (fastest knob, meaning-preserving when kept small).
    - Increase `humanizer_variance.max_ops_per_1000w` in small steps:
        - `0.5 -> 1.0 -> 1.5` (stop if the output starts to feel noisy or meaning shifts).
    - Use `--seed 0` (or `--seed` with no value) to randomize the seed for that run, so repeated runs do not converge on the same micro-edits.

3. Increase per-chunk variability targets (controller overlays).
    - Expand `humanization_controller.quantiles` to include more extremes, e.g.:
        - `[0.25, 0.5, 0.75] -> [0.15, 0.5, 0.85]` (or add `[0.10, 0.90]` for stronger variation).
    - If overlays feel too weak, increase `humanization_controller.range_pct` modestly:
        - `0.15 -> 0.20` or `0.25`.
    - Why: each chunk receives slightly different distribution targets, so the output can express variability across the document without relying on arbitrary randomness.
4. Give perturbations more "surface area" (optional).
    - Lower `chunking.max_input_tokens` so a long document becomes more chunks (more overlay samples, more variance opportunities).
    - If the input is short but you still want perturbations to have room, raise `chunking.min_chunks_when_perturbing` (for example, `2 -> 3`).
    - Tradeoff: more chunks means more LLM calls and more opportunities for coherence drift; chunk summaries help, but you still pay latency.
5. Last resort: increase model randomness.
    - Slightly increase `temperature` in `config.llm.json` (for example, `0.2 -> 0.3` or `0.4`).
    - Tradeoff: this increases global sampling variance, which can increase semantic drift risk. Prefer the bounded mechanisms above first.

Lexical Lists (`lexical_signals`, `lexical_avoidance`) *These limits control how many lexical signals and avoidance candidates are retained.*

- `lexical_signals.rare_words_limit` (integer): maximum number of rare words to include in `measurements.lexical_signals.rare_words`.
- `lexical_avoidance.rare_words_limit` (integer): maximum number of absent common words (from `config.common_words.txt`) to include in `measurements.lexical_avoidance.rare_words`.

Control Normalization (`controls_normalization`) *These options compress and de-duplicate control text to reduce prompt noise while preserving intent.*

- `controls_normalization.rewrite_policy.jaccard_threshold` (0–1): similarity threshold for considering two rewrite-policy clauses duplicates (lower = more aggressive de-dup).
- `controls_normalization.rewrite_policy.dedupe_on_subset` (boolean): treat clauses as duplicates when one clause is a strict subset of another.
- `controls_normalization.rewrite_policy.prefer_more_specific` (boolean): when near-duplicates are found, keep the clause with more unique tokens.

- `controls_normalization.rewrite_policy.compress_directives` (boolean): merge repeated `preserve` / `avoid` directives into a smaller number of interpretable clauses (reduces redundancy and token overhead).
- `controls_normalization.rewrite_policy.directive_verbs` (array): verbs that mark the start of new rewrite-policy clauses.
- `controls_normalization.rewrite_policy.stopwords` (array): stopwords ignored when comparing clauses for de-duplication.
- `controls_normalization.priority_order.token_pattern` (regex string): which items are allowed to survive normalization (default keeps short token-like priorities only).
- `controls_normalization.priority_order.dedupe_case_insensitive` (boolean): de-dup priorities ignoring case.
- `controls_normalization.priority_order.exclude_tokens` (array): drop these token-like entries even if they match the regex (the pipeline also drops generic `lexical`, `syntactic`, and `rhetorical` tokens by default).

Genre Detection (`fiction_detection`) *These heuristics determine fiction vs non-fiction handling, especially for quotation treatment.*

- `fiction_detection.quote_span_min` (integer): minimum multi-word quote spans required before classifying as fiction (lower = more likely fiction).
- `fiction_detection.quoted_ratio_min` (float 0–1): minimum fraction of words inside multi-word quotes to classify as fiction (lower = more likely fiction).
- `fiction_detection.quote_para_ratio_min` (float 0–1): minimum fraction of paragraphs starting with a quote to classify as fiction (lower = more likely fiction).
- `fiction_detection.quoted_ratio_force` (float 0–1): if quoted word ratio exceeds this, force fiction regardless of other signals.

Troubleshooting: non-fiction detected as fiction (quotes getting rewritten)

If the run header shows `Mode fiction (detected); quotes mutable` but your document is non-fiction and you want multi-word quotations preserved:

1. Quick fix (per run): force the mode explicitly.
    - `apply_fingerprint.py` : pass `--non-fiction`
    - `fingerprint_style.py` : pass `--non-fiction` (so profiling excludes multi-word quotations too) This is the safest option when the document is structurally "quote heavy" (transcripts, long epigraphs, interview Q/A, policy excerpts).
2. Persistent fix (tuning): raise the thresholds in `config.tunables.json` under `fiction_detection` . The current heuristic classifies as fiction when any of the following are true:
    - `quote_spans >= quote_span_min` AND `quoted_ratio >= quoted_ratio_min` , OR
    - `quote_para_ratio >= quote_para_ratio_min` AND `quoted_ratio >= quoted_ratio_min` , OR
    - `quoted_ratio >= quoted_ratio_force` (force-fiction guard).

    Recommended knobs (start here, then re-run and iterate):

    - If the document contains long quoted blocks (high quoted-word share): raise `quoted_ratio_force` first.
        - Typical change: `0.08 -> 0.12` (or `0.15` if the document is extremely quote heavy).
        - Why: this prevents "force fiction" from triggering purely because a non-fiction document contains many quoted words.
    - If you have many short, two-plus-word quotes ("scare quotes") but they are a small fraction of the document:
        - Raise `quoted_ratio_min` : `0.03 -> 0.05` .
        - Why: it makes the classifier require that quoted words occupy a meaningful share of the text before calling it fiction.
    - If you have many multi-word quote spans overall, but they are not dialogue:
        - Raise `quote_span_min` : `6 -> 10` (or `12`).
        - Why: it requires more distinct multi-word quote spans before the span-count signal can trigger fiction.
    - If the document has many paragraphs that begin with quotes (common in transcripts or formatted excerpts):
        - Raise `quote_para_ratio_min` : `0.20 -> 0.30` (or `0.40`).
        - Why: it reduces false positives when a non-fiction document uses quoted paragraphs as formatting rather than dialogue.

Chunking, Continuity, and Recovery (`chunking`) *These settings define chunk size/splitting, continuity summaries, and fallback behavior on invalid outputs.*

- `chunking.max_input_tokens` (integer): hard cap on input tokens per chunk (after prompt overhead). Lower values increase chunk count but reduce per-request latency and timeouts.
- `chunking.chunk_split_on` (`word`, `sentence`, or `paragraph`): primary unit for chunking. `sentence` is default. If a paragraph exceeds the token budget, it falls back to sentence splitting for that chunk; if a single sentence is still too long, it falls back to word splitting just for that chunk. Bullet/numbered list lines are treated as sentence units even without terminal punctuation.
- `chunking.chunk_summary.enabled` (boolean): when true, each chunk asks the LLM for a short rolling summary of the "gist so far" to carry into the next chunk. This helps maintain narrative/topic continuity across many chunks and is excluded from the final output.
- Retry optimisation: the chunk summary is requested on attempt 1 only, then reused across style/voice retries to reduce prompt tokens. Exception: if the first summary is empty or meta/task-focused, one refresh request is allowed on a later attempt. The final chunk does not request a summary.
- `chunking.chunk_summary.summary_words` (integer): target word count for the rolling summary (project config currently `50`; internal fallback default `25`). Keep small to minimise token overhead.
- `chunking.min_chunks_when_perturbing` (integer): enforce a minimum number of chunks when perturbations are enabled (humanizer variance or controller overlays), so variability has room to express.
- `chunking.recovery_split_max_depth` (integer): when the LLM repeatedly returns invalid output for a chunk, this controls how many recursive recovery splits may be attempted.
- `chunking.recovery_split_min_chars` (integer): minimum chunk size (in characters) before attempting recovery splitting; smaller chunks are preserved verbatim instead.
- `chunking.variance_aware.enabled` (boolean): when true, chunk sizes are scaled based on baseline variability (higher variance -> smaller chunks).
- `chunking.variance_aware.sentence_stdev_ref` (float): reference sentence-length stdev for scaling.
- `chunking.variance_aware.paragraph_burst_ref` (float): reference paragraph burstiness for scaling.
- `chunking.variance_aware.min_factor` (float): minimum multiplier applied to `max_input_tokens` when variance is high.
- `chunking.variance_aware.max_factor` (float): maximum multiplier applied to `max_input_tokens` when variance is low.

Style/Voice Retry Budgets (`style_retry`) *These budgets cap compliance retries and forced-person voice retries for each chunk.*

- `style_retry.enabled` (boolean): enable/disable the actionable feedback retry pass after measuring style compliance.
- `style_retry.threshold` (0–1): retry when compliance score is below this threshold (project config currently `0.60`; internal fallback default `0.75`). Lower values trigger fewer retries (more permissive); higher values trigger more retries (stricter). `0.0` effectively disables threshold-based retries, while `1.0` retries unless the output is nearly perfect.

- `style_retry.max_retries` (integer): maximum additional retry passes for the style loop after the initial attempt.
- `style_retry.voice_max_retries` (integer): maximum retry passes for the forced-person voice loop (`--1st-person` / `--2nd-person` / `--3rd-person`). If omitted, it inherits `style_retry.max_retries` .

What `style_retry.threshold` actually measures

The "compliance score" is a local, interpretable similarity score in the range 0 to 1 computed after each chunk rewrite. It compares the output chunk's measured stylometric signals against the fingerprint's measured corpus signals and aggregates the result:

- `1.0` means the output chunk's measurements closely match the fingerprint measurements across the scored sections.
- `0.0` means large divergence across one or more scored sections.

The score is based on measurements of author-voice text only (blockquotes/references/citations are excluded, and in non-fiction mode multi-word quotations are preserved and excluded from measurement). It is not a meaning-preservation score; it is only used to decide whether to spend additional LLM calls to better match the fingerprint.

Under the hood, the compliance score is a weighted average of section-level similarities. Each section contributes a 0-1 subscore computed from an explicit distance:

- Histogram sections (sentence length, paragraph length): distance is total variation, `d = 0.5 * sum_i |p_i - q_i|` (ranges 0-1), then `score = 1 - d` .
- Scalar/rate sections (punctuation rates, stance signals, rhetoric moves, epistemic profile, syntax texture, etc.): distance is a clipped relative error, `d = |out - target| / max(|target|, 1)` , then `score = 1 - clip(d, 0, 1)` . Multi-field sections average their per-field distances before scoring.

Section weights come from `fingerprint.validators.weights` when present; otherwise, sections are averaged equally. In verbose logs, the per-attempt `score=X.XXX` value is this aggregate compliance score.

How to pick a threshold:

- Treat it as a "good enough" gate, not a guarantee of perfection. Because compliance is computed per chunk, short chunks and high-variance writing tend to have noisier measurements.
- If most chunks plateau below your threshold even after retries, the threshold is usually too strict for that fingerprint/input pair. Lower it (for example, `0.75 -> 0.65`) or reduce the number of retries to cap cost.
- If you care about only a subset of signals, adjust `fingerprint.validators.weights` to emphasize them, rather than pushing the global threshold very high.

Section Restoration (`section_restore`) *These thresholds control recovery of missing sections after rewriting.*

- `section_restore.enabled` (boolean): enable/disable restoration of missing sections detected after rewriting.
- `section_restore.max_restore_sections` (integer): maximum number of missing sections to restore (0 disables restoration).
- `section_restore.heading_similarity_threshold` (0–1): fuzzy heading match threshold for considering a rewritten heading “present”.
- `section_restore.signature_similarity_threshold` (0–1): content-signature similarity threshold for matching a section by its opening content.
- `section_restore.signature_min_overlap` (integer): minimum number of overlapping signature tokens required for a content match.

Deterministic Redundancy Post-Processing (`postprocess_redundancy`) *These controls reduce repetitive AI-like structure after chunk stitching while preserving semantic content.*

- `postprocess_redundancy.enabled` (boolean): master switch for deterministic anti-redundancy pass.
- `postprocess_redundancy.paragraph_dedupe.enabled` (boolean): enables near-duplicate prose block removal.
- `postprocess_redundancy.paragraph_dedupe.min_words` (integer): minimum block length for dedupe eligibility.
- `postprocess_redundancy.paragraph_dedupe.similarity_threshold` (0.8–1.0): canonical similarity threshold above which a prose block is treated as duplicate.
- `postprocess_redundancy.paragraph_dedupe.lookback_blocks` (integer): how many recent blocks to compare against.
- `postprocess_redundancy.paragraph_dedupe.max_drop_ratio` (0.01–0.5): safety cap on removable block fraction in one document.
- `postprocess_redundancy.list_density.enabled` (boolean): enables unordered-list run throttling.
- `postprocess_redundancy.list_density.min_run_length` (integer): minimum contiguous unordered bullets required before grouping is applied.
- `postprocess_redundancy.list_density.group_size` (integer): number of bullet items merged into each grouped bullet.
- `postprocess_redundancy.list_density.joiner` (string): separator used when grouping multiple bullet items.

Sanity Check Warnings (`sanity_checks`) *These percentage thresholds trigger review warnings when output size drifts materially from input.*

- `line_count_warn_pct` (%): if the output line count changes by this percentage or more, a console warning is emitted to review for missing or expanded content.
- `word_count_warn_pct` (%): if the output word count changes by this percentage or more, a console warning is emitted to review for missing or expanded content.
- `paragraph_count_warn_pct` (%): if the output paragraph count changes by this percentage or more, a console warning is emitted to review for missing or expanded content.

All thresholds are conservative defaults. Lowering a threshold increases the likelihood of a conflict (more rules dropped). Raising a threshold makes the humanizer rules more permissive.

Retry semantics (clear execution order)

- `config.llm.json:max_retries` : Applies to each HTTP call to the LLM endpoint (`chat_completions`). It retries transport/transient failures (timeouts, 429/5xx, connection errors) with exponential backoff.
- `config.llm.json:backoff_base_seconds` / `backoff_max_seconds` : Controls the sleep schedule for the transport retries above.
- `config.llm.json:max_retries` (second use in apply path): The chunk-rewrite loop also uses this budget to recover invalid model payloads (for example missing/empty `final_markdown`), re-calling the model before falling back to split recovery or verbatim preserve. Before invoking LLM repair, the apply parser attempts deterministic repair for common malformed JSON string quoting in `final_markdown` , `chunk_summary` , deviation `reason` fields, and `self_check.notes` .
- `style_retry.enabled` : Enables/disables threshold-based style retry logic entirely.
- `style_retry.threshold` : If local compliance score is below threshold, a style retry pass is attempted (subject to `style_retry.max_retries`). Retry prompts include prioritized failed constraints, actual vs target measurements, response diagnostics such as long sentences/repetition, and explicit preservation instructions.
- `style_retry.max_retries` : Sets retry budget for the style loop (additional passes after the base attempt).
- `style_retry.voice_max_retries` : Sets retry budget for the voice loop when forced-person mode is enabled. If absent, uses `style_retry.max_retries` .
- Forced-person voice repair: Before consuming a full voice retry, the apply path first attempts high-confidence deterministic sentence repairs, then an optional sentence-only LLM repair for remaining violating sentences. Full chunk retry is used only if those local/micro repairs do not satisfy the forced-person check.
- `humanization_controller.max_feedback_retries` : Caps only how many style retries carry controller-overlay delta feedback. It does not increase retry count; it only changes feedback content.

Practical counting model (per chunk)

- Base attempt is always 1.
- Style-only mode: up to `1 + style_retry.max_retries` attempts.
- Forced-person mode:
  - Before a full voice retry, sentence-level deterministic and micro-LLM repairs may resolve pronoun violations without resending the whole chunk.
  - Voice loop can consume up to `style_retry.voice_max_retries` additional attempts (or `style_retry.max_retries` if voice cap is unset).
  - Style loop can then consume up to `style_retry.max_retries` additional attempts.
  - Total attempts can therefore approach `1 + voice_cap + style_retry.max_retries` (plus transport retries inside each call).
- Each attempt may itself contain up to `1 + config.llm.max_retries` transport attempts at HTTP level.

Console output is intentionally compact and table-oriented: normal runs print a short run header, mode table, and one consolidated final result table for artifacts, count deltas, and metrics. In verbose mode, grouped settings, retry budgets, token budgets, chunk token min/avg/max, metric scores, concise retry lines, and best-attempt replacement details are shown in compact colorized box-drawing tables where useful. Table headers use a very dark blue background with bright text. Compatible table rows stream under one header; double rules separate headers from data, single rules separate logical rows, and packed setting/value groups use double vertical separators between groups. Non-table status lines close the table so the next table reprints headers. Long cells wrap onto aligned continuation lines. Set `NO_COLOR=1` or `CLICOLOR=0` to disable ANSI colors.

Global avoid list: `config.avoid.txt`

If present, `config.avoid.txt` provides a hard “never use” list. Each non-empty line is treated as a word or short phrase to avoid. Lines may include comments after `#`, and blank lines are ignored. **Entries are treated literally (no spelling normalization)**, so include any local spelling variants you need. The list is:

- Injected into fingerprinting as hard lexicon avoids
- Merged into `lexicon.avoid_words` during application (even if the fingerprint does not include them)

Organizational bans, regulatory requirements or personal preferences may take precedence over the author’s stylistic choices.

The algorithmic common-word omissions from `config.common_words.txt` populate `lexicon.avoid_words_soft` instead (soft guidance).

Entity blacklist: `config.entity_blacklist.txt`

If present, `config.entity_blacklist.txt` provides a list of entity names (people, places, organizations) that should be excluded from common-phrase extraction. This helps prevent proper-name phrases (e.g., “new york”, “microsoft”, “jane doe”) from dominating `common_phrases`. Lines may include comments after `#`, and blank lines are ignored. Multi-word names are supported. Single-word names shorter than 3 characters are ignored.

Usage

1. Building a Style Fingerprint

Begin by creating a compressed archive of your writing corpus:

```
tar -czf my_corpus.tar.gz essays/ notes/ drafts/
```

Fingerprinting is performed as follows:

```
python fingerprint_style.py \
  -a my_corpus.tar.gz \
  -o my_fingerprint.json \
  --profile-id "me_style_v1" \
  --author-name "Me"
```

Alternatively, use the wrapper script:

```
./scripts/fingerprint_style.sh \
  -a my_corpus.tar.gz \
  -o my_fingerprint.json \
  --profile-id "me_style_v1" \
  --author-name "Me"
```

To specify a non-default configuration path, pass `-c/--config`. If `--profile-id` or `--author-name` are not provided, they default to the output filename without the `.json` extension (for example, `my_fingerprint`). Normal output is compact; progress details are enabled with `-v/--verbose`. To inspect LLM JSON responses, pass `--debug`; each parsed response is emitted to stderr as a jq-formatted JSON object.

By default, common phrases are validated using an additional LLM pass to filter out OCR errors and citation fragments. This can be disabled via `--no-phrase-validation`.

Large corpora are automatically chunked according to `max_prompt_tokens`; override this with `--max-prompt-tokens`.

The process will:

- Extract the archive
- Measure stylistic statistics (excluding blockquotes, reference sections, footnotes, boilerplate notices, and inline citations)
- Send measurements and excerpts to the LLM
- Produce `my_fingerprint.json`

---

2. Applying a Fingerprint

To rewrite a Markdown file in your style:

```
python apply_fingerprint.py \
  -f my_fingerprint.json \
  -i draft.md
```

Alternatively, use the wrapper script:

```
./scripts/apply_fingerprint.sh \
  -f my_fingerprint.json \
  -i draft.md
```

Specify a non-default configuration path with `-c/--config`. Normal output is compact; progress details are enabled with `-v/--verbose`. To inspect LLM JSON responses, pass `--debug`; each parsed response is emitted to stderr as a jq-formatted JSON object. `-f/--fingerprint` appends `.json` if no extension is given. Long inputs are chunked automatically based on `max_prompt_tokens`; override this with `--max-prompt-tokens`.

Style compliance is scored locally. If the score falls below the threshold, the system performs bounded retry passes according to `style_retry` tunables (or CLI overrides) and produces actionable feedback between attempts: top failed constraints, actual vs target measurements, response diagnostics, and preservation instructions (disable with `--no-style-retry`, adjust with `--style-retry-threshold` or `--max-style-retries`).

If `general-guidelines.md` is present in the repository root or next to the scripts, its humanization rules (adapted from softaworks/agent-toolkit by @leonardocouy) are parsed with an LLM by default. Deterministically conflicting guidance (based on fingerprint signals such as em-dash rate, hedging or first-person use) is dropped before prompting. This introduces one additional LLM call when enabled. Parsed rules are cached in `humanizer_rules.cache.json` next to the scripts and are only re-parsed

when `general-guidelines.md` changes. LLM parsing can be disabled via `--no-humanizer-llm-parse`, or the guidelines can be disabled entirely via `--no-humanizer-guidelines`.

Pronoun override flags let you force the narrative voice regardless of the fingerprint: `--1st-person`, `--2nd-person`, or `--3rd-person`. When `--3rd-person` is combined with `--author-name AUTHOR_NAME`, the LLM is instructed to occasionally use that name instead of he/she/they where it reads naturally, including once after each Markdown heading when the following prose naturally refers to the author/narrator.

Interpreting forced-voice logs (verbose mode)

When a pronoun override is active, each chunk is checked for "wrong-person" pronouns used in subject-like roles. If violations are detected, the system first tries deterministic sentence-level repair and then sentence-only LLM repair before spending a full chunk voice retry.

You will see log lines like:

```
  - Chunk 1/2: sentence-level voice repair 2 sentence(s)
  - Chunk 1/2: voice retry 1/1 (mode=third; allowed_count=105;
violations[first_person=1, second_person=8];
ignored[first_person_non_subject=4, second_person_non_subject=4])
```

How to read this:

- `voice retry 1/1`: the current attempt failed the forced-voice check, and the system is about to spend its 1st (and final) voice retry. `voice_max_retries=N` means "up to N additional voice retries after the initial attempt".
- `mode=third`: the forced voice mode for this run (`first`, `second`, or `third`).
- `allowed_count=105`: how many pronoun tokens match the target mode in the chunk's author-voice text. This is a coverage signal, not a violation signal. It is used as a tie-breaker when selecting the best attempt (more target-voice pronouns is better when violations are equal).
- `violations[...]`: disallowed pronouns that look like grammatical subjects (approximate heuristic, not a full parser). In this example, third-person voice is being forced, but the model still produced some first-person ("I/we") and second-person ("you") subject-like uses.
- `ignored[...]`: disallowed pronouns that were detected but treated as non-subject roles and therefore not counted as voice violations. This is how object/complement cases stay legal. Example: with `--1st-person`, "I had to help him" should not count "him" as a voice violation.

Notes and common gotchas:

- The check runs on “author-voice” text only (blockquotes/references/citations are excluded; and in non-fiction mode, multi-word quotations are masked/preserved), so the voice loop is not usually driven by quoted material unless you are in fiction mode.
- If you see mostly `second_person` violations while forcing `first` or `third`, that often indicates direct address (“you …”) is persisting (commonly in dialogue or instructional prose).
- If voice retries are exhausting frequently, adjust `style_retry.voice_max_retries` (voice budget) independently of `style_retry.max_retries` (style budget). Voice and style loops are separate: voice retries are triggered by pronoun violations; style retries are triggered by compliance score falling below the threshold.

`--local-spelling {none|canadian|australian|british|us}` overrides both split spelling settings for a single run.
`--local-spelling-llm {none|canadian|australian|british|us}` overrides only `humanizer_mandatory.force_local_spelling_LLM`.
`--local-spelling-rules {none|canadian|australian|british|us}` overrides only `humanizer_mandatory.force_local_spelling_rules`.
`--perplexity {default|low|medium|high|extreme}` overrides tunables `perplexity_level` for a single run.
`--roster [int]` enables multi-model chunk routing from `config.llm.roster.json`; with an integer seed, each roster cycle is shuffled deterministically and no model repeats until all entries are used.
`--seed [int]` overrides `humanizer_variance.seed` for a single run (`0` or omitted value = random seed). The override also drives controller-overlay sampling so both systems remain aligned. `--query perplexity` (or `--query=perplexity`) prints the configured perplexity level on a single line and exits; other arguments are ignored.

Example (British spelling with mixed contexts):

Input:

```
The program compiled quickly. The program airs tonight on the public
broadcaster.
```

Output with `--local-spelling british`:

```
The program compiled quickly. The programme airs tonight on the public
broadcaster.
```

Rules used for the example (from `config.local_spelling_rules.json`):

```
{
  "id": "program_programme",
  "variants": {
    "us": "program",
    "canadian": "program",
    "british": "programme",
    "australian": "programme"
  },
  "avoid_if": {
    "any": [
      "algorithm",
      "api",
      "application",
      "binary",
      "code",
      "compile",
      "compiler",
      "debug",
      "programming",
      "software"
    ]
  },
  "apply_if": {
    "any": [
      "broadcast",
      "episode",
      "public",
      "radio",
      "television",
      "tv"
    ]
  },
  "window": 6
}
```

Rule precedence (local spelling):

- **Context rules first** (`context_variants`): evaluated before any direct/suffix rules.
  - `block_if` wins (no change).
  - `apply_if` must be satisfied (otherwise the rule is skipped).
  - `avoid_if` blocks the rule if matched.
- **Direct variants** next (`direct_variants`), then **suffix variants** (`suffix_variants`), then **double-L inflections**.

So in the example, the first "program" sees `compile` nearby and is blocked by `avoid_if` , but the second "program" sees `broadcast/public` and is converted to "programme."

Lexical avoidance vs. local spelling: avoidance checks are normalized to **US spelling** for matching, then the **output** is normalized to the selected local spelling. This avoids missing soft avoids when the output uses local variants (for example, "colour" vs "color"). Fingerprints store lexicon entries in a **US-normalized baseline** so cross-profile comparisons are consistent, and local spelling is applied only at rewrite time.

Embedded BASE64 images are removed from prompts to avoid excessive token usage and re-inserted into the rewritten output. Blockquotes, reference sections, footnotes, boilerplate notices, and inline citations are preserved verbatim and excluded from style transfer.

Outputs:

- `draft.md.styled.md` : rewritten text
- `draft.md.styled.md.deviations.json` : any rule conflicts or deviations
  - When `--metrics` is enabled, includes `humanization_metrics` for **both input and output**, with heuristic quantitative scores plus an aggregate 0–100 score (`aggregate_score_100`).
  - Metrics include: lexical diversity (TTR, Herdan's C, Guiraud's R, Maas TTR), Yule's K, Simpson's D, repetition inverse, sentence/paragraph burstiness, punctuation variety/entropy, function-word entropy and KL-inverse vs fingerprint, sentence-length JS-inverse vs fingerprint, character trigram entropy, and average word length.

---

3. Local HTTP API

Run the local API (HTTP only; local deployment intent):

```
python fingerprint_api.py --host 127.0.0.1 --port 8765
```

You can also set the API port with `--api N` :

```
python fingerprint_api.py --host 127.0.0.1 --api 8765
```

Wrapper script:

```
./scripts/fingerprint_api.sh --host 127.0.0.1 --port 8765
```

`--api` is also accepted by the wrapper because all args are passed through unchanged.

The API stores fingerprints in a repository-root subdirectory (`fingerprint_store/`) as:

- `<guid>.fingerprint.json`
- `<guid>.meta.json`

Methods:

- `POST /make` : accepts text and creates a fingerprint; returns a GUID
- `POST /apply` : accepts GUID + text and returns rewritten text + deviations
- `POST /rate` : accepts GUID + text and returns style-match probability
- `POST /similarity` : accepts two GUIDs and returns fingerprint similarity diagnostics

Example: `make`

```
curl -s http://127.0.0.1:8765/make \
  -H "Content-Type: application/json" \
  -d '{"text":"Sample corpus text for style extraction."}'
```

Example: `apply`

```
curl -s http://127.0.0.1:8765/apply \
  -H "Content-Type: application/json" \
  -d '{"id":"<GUID_FROM_MAKE>","text":"New text to rewrite."}'
```

Example: `rate`

```
curl -s http://127.0.0.1:8765/rate \
  -H "Content-Type: application/json" \
  -d '{"id":"<GUID_FROM_MAKE>","text":"Candidate text segment."}'
```

Example: `similarity`

```
curl -s http://127.0.0.1:8765/similarity \
  -H "Content-Type: application/json" \
  -d '{"id_a":"<GUID_A>","id_b":"<GUID_B>"}'
```

Swagger/OpenAPI artifacts:

- `api/swagger/openapi.yaml`
- `api/swagger/openapi.json`

The API also serves:

- `GET /health`
- `GET /openapi.yaml`
- `GET /openapi.json`

`rate` probability details:

- Base score: local style compliance score (`0..1`) from the same measurement layer used by rewrite retries.
- Calibration: logistic mapping centered at the style threshold (default: `config.tunables.json -> style_retry.threshold`).
- Reliability shrinkage: short segments are shrunk toward `0.5` to reflect lower evidence, and a 90% confidence interval is returned.

`similarity` method details:

- Compares two stored fingerprints directly (no rewrite step).
- Uses interpretable per-component signals (histogram/divergence, rate similarity, function-word distribution, lexical overlap).
- Returns:
  - overall `similarity_score` (`0..1`)
  - `distance_score` (`1 - similarity`)
  - per-component metrics and top differences
  - coverage and confidence hints to show when the comparison is underpowered.

---

4. API GUI Harness

The GUI harness provides a no- `curl` way to try all API endpoints:

- `POST /make`
- `POST /apply`
- `POST /rate`
- `POST /similarity`
- utility calls: `GET /health` , `GET /openapi.json` , `GET /openapi.yaml`

Launcher script (runnable from anywhere):

```
./scripts/fingerprint_api_harness.sh
```

How port selection works:

- The launcher defines a constant at the top of the script: `API_PORT=8765`
- It starts the harness with `--api "${API_PORT}"` by default
- You can still override it at runtime, for example:

```
./scripts/fingerprint_api_harness.sh --api 9000
```

Typical flow:

1. Start the API server (`fingerprint_api.py`) on a chosen port.
2. Start the harness script.
3. In the harness top bar, confirm host/port and click `Apply Host/Port` .
4. Use each endpoint tab to submit requests and inspect JSON responses in the Response panel.

The harness uses Tkinter (`python3-tk` on many Linux distros) plus `requests` .

Visualize a Fingerprint

Generate an HTML dashboard to review key fingerprint settings and measurements:

```
python show_fingerprint.py path/to/fingerprint.json -o
fingerprint_dashboard.html
```

Use `--open` to launch the dashboard in your browser.

Wrapper script (can be called from anywhere):

```
./scripts/show_fingerprint.sh path/to/fingerprint.json -o
fingerprint_dashboard.html --open
```

Output Files

Style Fingerprint (`*.json`)

Contents include:

- `metadata` : corpus and extraction information
  - `metadata.corpus.document_count` : number of corpus documents
- `metadata.corpus.documents` : per-document metadata (path, title when available, size, language/locale, genres, time range)

- `measurements` : raw statistical signals (including `orthography_signals.spelling_variant` , paragraph rhythm, `lexical_signals.rare_words` , and `lexical_avoidance.rare_words` derived from a built-in common-words list)
- `targets` : stylistic constraints and distributions (including optional persona pronoun preferences)
- `lexicon` : preferred and avoided words and phrases (`avoid_words` = hard avoids, `avoid_words_soft` = soft avoids)
- `templates` : syntactic and rhetorical patterns
- `controls` : strictness, priority ordering, and optional humanizer variance (seeded, bounded micro-variation)
- `validators` : scoring weights and checks
- `derived_instructions` : compiled prompts for generation and rewriting

This file is human readable, editable, version controllable and reusable across projects.

API Fingerprint Store (`fingerprint_store/`)

When using `fingerprint_api.py` , fingerprints are tracked by GUID in a repo-root subdirectory:

- `fingerprint_store/<guid>.fingerprint.json` : stored fingerprint artifact
- `fingerprint_store/<guid>.meta.json` : metadata for source/run tracking

API Specification Artifacts

- `api/swagger/openapi.yaml` : primary OpenAPI specification
- `api/swagger/openapi.json` : JSON-form OpenAPI specification

Testing

Lightweight smoke tests are located in `tests/` and exercise the full pipeline using small fixtures when LLM tests are enabled. By default, only the regression suites run (no LLM calls).

To run the smoke test:

```
./tests/run_smoke.sh
```

Artifacts are written to `tests/_artifacts/` (gitignored).

The v1.1.0 regression suite (no API calls) is found in `tests/test_v1_1_0_regression.py` and is automatically executed by `run_smoke.sh` . It can also be run directly:

```
./tests/run_v1_1_0_regression.sh
```

The v1.5.x regression suite (no API calls) is found in `tests/test_v1_5_X_regression.py` and is also executed by `run_smoke.sh`:

```
./tests/run_v1_5_X_regression.sh
```

The v1.7.x regression suite (no API calls) is found in `tests/test_v1_7_X_regression.py` and is also executed by `run_smoke.sh`:

```
./tests/run_v1_7_X_regression.sh
```

The v1.8.x regression suite (no API calls) is found in `tests/test_v1_8_X_regression.py` and is also executed by `run_smoke.sh`:

```
./tests/run_v1_8_X_regression.sh
```

An LLM connectivity check and the end-to-end fingerprint/apply smoke path can be enabled by passing `--llm-tests` to `run_smoke.sh`, which runs `tests/test_llm_smoke.py` using the same `config.llm.json`.

Style Model Schema

The schema models several layers:

- Orthography and formatting
- Punctuation signature
- Sentence rhythm and clause structure
- Paragraph architecture
- Lexical preferences
- Semantic tendencies
- Rhetorical moves
- Persona and stance (including pronoun preferences)

Supported features include:

- Target values with tolerance ranges
- Histograms rather than single averages
- Hard versus soft constraints
- Priority-ordered enforcement

This enables interpretable, controllable generation rather than opaque imitation.

Ethics and Intended Use

The project is intended for:

- Personal writing consistency
- Author self-modelling
- Editing assistance
- Long-term voice preservation

It is not intended for:

- Impersonating living authors without consent
- Passing generated text off as another person
- Circumventing authorship or attribution

Recommended practice:

- Use only your own writing or licensed/public-domain corpora
- Set `do_not_imitate_living_author = true` in controls
- Clearly label AI-assisted outputs when appropriate

Roadmap

Planned extensions:

- [ ] JSON Schema validation (`jsonschema`)
- [ ] HTML / PDF corpus ingestion
- [ ] Streaming support for long-running LLM calls
- [ ] Style similarity scoring CLI
- [ ] Batch rewriting
- [ ] Fine-grained constraint toggles
- [ ] Visualisation of stylistic distributions

References

Core research areas:

- Stylometry / computational stylistics
- Authorship attribution
- Text style transfer
- Interpretable controllable generation

Representative terms:

- *Stylometric profiling*
- *Author-conditioned text generation*
- *Feature-based style modeling*
- *Constraint-augmented rewriting*

Acknowledgments

Inspired by:

- Stylometric authorship research
- Controllable text generation literature
- Practical needs for long-term personal voice consistency

---

stylometric-transfer: explicit style models for interpretable author voice transfer